Week 1 Writeup - Identify the Problem Statement and Dataset
Langley, Jack, Chan

Problem Statement

With the amount of music being released each week, people tend to get stuck listening to the same songs or artists over and over due to the amount of time it takes to explore and find similar music that matches what you like. Different music platforms like Apple and Spotify do a good job with recommendations, but they often prioritize matching genres or suggest what other users are listening to. This typically leads to missing out on music that matches the style or meaning behind a song that you love.

For this project, we wanted to take a more creative and personalized approach - the goal is to develop a recommendation system that suggests songs based on their semantic similarity to a given track. The idea is to capture how songs feel similar - not just because they're both "pop" or "indie", but because they share similar lyrics, tempo, emotion, or energy. For example, if someone's listening to a slow breakup song, we can recommend others with that same lyrical theme and musical type - even if the artist or genre is different.

Project Value

This kind of model could make music discovery more engaging, relevant, and personalized. Instead of relying on popularity or genre, listeners get recommendations that truly match the feel of what they like at the time. It can also help discovery of less popular songs and artists that would otherwise be overlooked, which gives platforms a way to boost discovery and user satisfaction at the same time!

Some great applications for this include automatically creating playlists that make sense mood-wise, helping users discover new artists that match their taste, and making music discovery more diverse, and enjoyable.

Calculation of the potential economic value

- **Document and footnote your assumptions**
    - Our model will increase Spotify's value by increasing user engagement, improving retention, and driving conversions from free subscribers to paid subscribers.
    - Using Spotify's Q1 report, we assume there are 317M free users and 239M paid subscribers.
    - Assuming Annual Revenue / User of $0.50/mo for free users (driven by ads) and $5.00/mo for paid users
    - Assuming $0.01/min listened in ad revenue and the LTV of the average user at $30
    - Assuming our model contribution increases free users listening time by an average of 2 minutes/user/day and improves overall retention by 0.5% annually
    - Also assumes it converts 0.1% of free users to paid subscribers annually
    - Costs $5M annually to develop, maintain and run at full scale
- **Show how you calculated the value**
    - Increased ad revenue
        - Extra 2 min/day * 30 days * $0.01 = $0.60 / user / month
        - Likely too high, will assume real number is ~10% after adjusting for skips, ad loading time and other factors = $0.06 / user / month
        - 317M users * $0.06 / user / month * 12 months = $228.24M
    - Converted paid subscribers
        - 0.1% conversion rate * 317M free users * $5.00 / mo * 12 months = $19.02M
    - Reduced Churn
        - 0.5% less churn * 556M total users * $30 / user = $83.40M
    - Overall
        - $228.24M + $19.02M + $83.04M - $5M = **$325.3M / year**

Project Plan

Below is a rough project plan for what steps to follow each week based on the syllabus and our project needs to stay on track through these 13 weeks.

Week 1 - Project Setup

- Define a clear problem - Connect similar songs through semantic similarity
- Specify project goals and value
- Finalize dataset
- Create GitHub repo and set up JupyterHub
- Submit initial written report

Week 2 - Data Loading

- Load scraped song data into Jupyter
- Verify schema (potential columns like lyrics, genre, release date, popularity)
- Inspect data types, completeness, and integrity
- Submit ingestion notebook and report
- Push to GitHub

Week 3 - Exploratory Data Analysis (EDA)

- Visualize trends in lyrics, genre, artist, and sentiment
- Profile key fields (ex. average word count, common words, genre distribution)
- Identify relationships (for example, genre vs. popularity, release year vs. sentiment)
- Submit updated notebook and report
- Commit to GitHub

Week 4 - Data Prep

- Clean text data such as removing stopwords
- Convert dates, encode genres/artists, normalize metrics
- Handle missing values or anomalies
- Document preprocessing steps
- Push updates to GitHub

Week 5 - Feature Engineering

- Create features (possibly - word count, sentiment score, genre dummy vars, tempo)
- Aggregate by artist/genre if needed
- Evaluate how engineered features capture semantic similarity between songs
- Submit code and report
- Update GitHub

Week 6 - Baseline Model (Simple)

- Implement a basic similarity approach
- Use default parameters and core textual features (for example - lyrics, song titles, genres)
- Evaluate similarity results using example queries or known similar songs
- Document findings, assumptions, and initial limitations
- Push all to GitHub

Week 7 - More Complex Model Dev.

- Generate richer song embeddings using advanced NLP features (sentiment scores, genre context)
- Apply unsupervised models (K-Means, hierarchical clustering, or similarity-based nearest neighbors)
- Compare with baseline similarity results from Week 6
- Visualize clusters or similarity graphs
- Document findings and push to GitHub

Week 8 - Most Complex Model Dev.

- Try deep learning (e.g., LSTM or Transformer-based lyric classification) or fine-tuned NLP models
- Conduct hyperparameter tuning to refine embeddings or clustering thresholds
- Focus on maximizing semantic similarity and interpretability of results
- Document methodology, outcomes, and technical challenges
- Push all updates to GitHub

Week 9 - Model Comparison and Selection of Final

- Evaluate all approaches using both quantitative metrics and qualitative analysis (are retrieved songs meaningfully similar?)
- Justify final method selection based on interpretability, performance, and alignment with project goals
- Summarize comparisons, trade-offs, and conclusions
- Push results to GitHub

Week 10 - Data Centric AI

- Augment or filter dataset?
- Re-run model to assess improvement from better data
- Reflect in written report
- Merge updates to GitHub

Week 11 - Model Explainability and Ethical Considerations

- Analyze potential bias (ex. genre, gender, era biases in popularity predictions?)
- Start peer feedback partner selection
- Document ethical considerations in report
- Push changes to GitHub

Week 12 - Deployment and Monitoring Plan

- Save final model
- Design a monitoring plan
- Clean and finalize repo
- Push all to GitHub

Week 13 - Wrap Up

- Finalize all notebooks, reports, visuals
- Make sure GitHub repo has clear README, structure, and references
- Submit final project (notebook + report)

Data Section

The songs in our dataset were initially “randomly” chosen through continuously searching tunebat.com by different song duration ranges. Tunebat is one of the few websites that has publicly available song metric data, including numerous song features like BPM, Key, Camelot, Popularity, Danceability, Energy, and Happiness. These metrics will be used as auxiliary features in the latent space of all songs that accompany the main feature which is the lyrics. From this repeated searching process, duplicates were removed and a broad array of artists were included, making sure no artist was too over-represented in the set.

Lyrics for all songs were then gathered from genius.com and attached to the initial features. At this point, we had a dataset of over 1,700 English language songs across all genres that can be found on Spotify. This set can be expanded at any time to include a more well-rounded set, however for now we feel comfortable with the extensivity of the data. Slight data preprocessing was done to ensure that the songs in the set were in the English language and no remixes were included.

Next steps for further refining the dataset include lyrics cleaning by removing punctuation and line breaks to make the tokenization process simpler. We will also have to consider some ethical / moral questions like if swear words and other derogatory terms should be included in the lyrics or not.

GitHub repo link: https://github.com/jmetz1313/spotify-project

Model Section

As we mentioned above, the goal is to develop a recommendation system that suggests songs based on their semantic similarity to a given song. This system would ingest key metadata, including lyrics, artist name, BPM, and other relevant attributes, to analyze and measure the underlying meaning and characteristics of different songs. The core challenge lies in effectively defining and quantifying the semantic relationships between tracks to provide meaningful recommendations that align with user preferences.

Possible Types of Modeling Approaches

(A) Unsupervised Learning Objective: Cluster or embed songs into a semantic feature space without labels. Techniques:

- Word2Vec / Doc2Vec / BERT Embedding: Sentence embedding based on lyrics
- t-SNE / PCA: Dimensionality reduction and visualization
- K-Means, Hierarchical Clustering: Clustering similar songs
- UMAP + DBSCAN: Flexible clustering based on semantic distance

(B) Supervised Learning Assumption: If labeled pairs of songs indicate whether they are similar or not, supervised learning can be applied. Problem Types:

- Binary Classification: Predict if two songs are similar (1) or not (0)
- Multi-class Classification: Categorize into multiple similarity levels
- Regression: Predict a semantic similarity score (0–1) between songs

(C) Recommendation System Perspective: A hybrid approach can be a practical solution.

- Content-based Filtering: Utilizes metadata such as lyrics, BPM, key, and mood.
- Collaborative Filtering: Based on user preferences and listening history
- Deep Learning (e.g., Siamese Networks): Learn embeddings based on similarity

Since the data lacks labels, the most natural initial approach is to begin with unsupervised learning, allowing the system to cluster or embed songs based on their semantic features without predefined classifications. As the project evolves, labeled pairs of similar songs can be collected, enabling a transition to supervised learning for more precise recommendations based on structured similarity assessments.

Since our project involves identifying songs that are semantically similar based on lyrics and metadata, the initial modeling will utilize unsupervised learning techniques, including embedding generation and clustering. If we later obtain labeled pairs of songs indicating similarity, we can extend our approach to supervised classification or regression models to refine our recommendations. A hybrid recommendation system combining content-based and semantic modeling will also be considered.